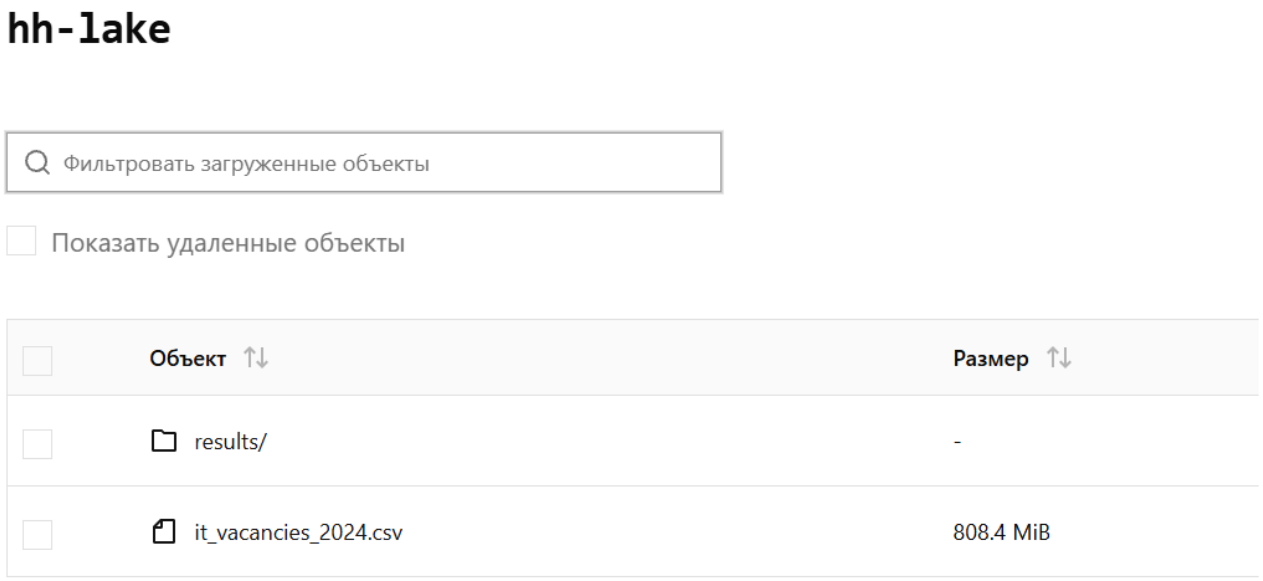
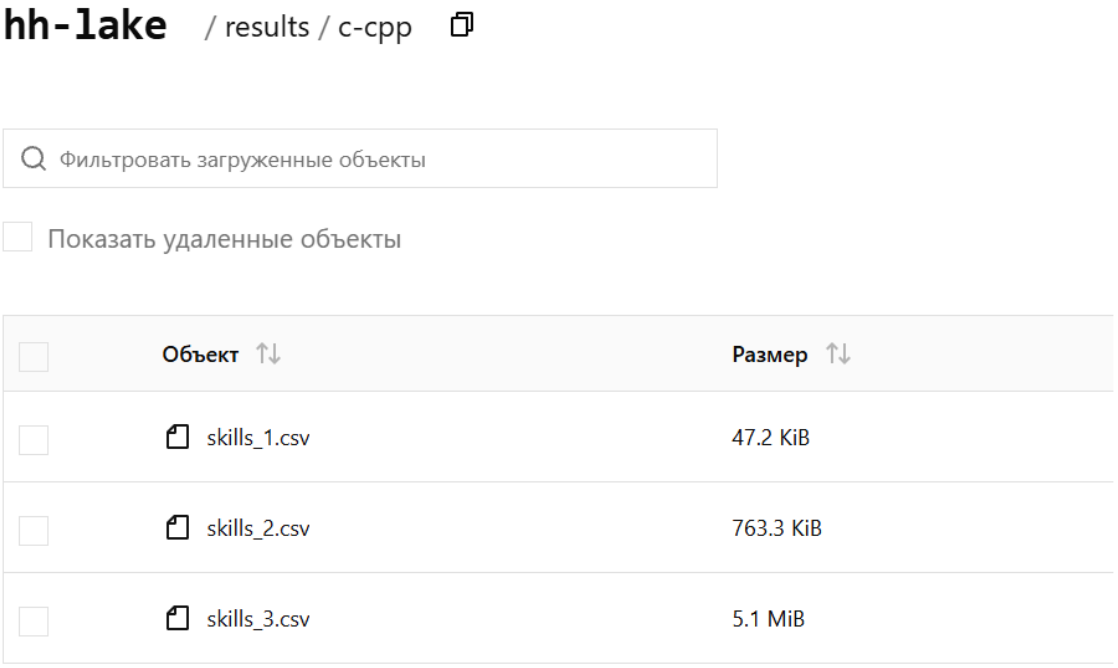
Технология MapReduce

Вариант: Программист, разработчик на C и C++

Объектное хранилище и исходные данные

Для отбора вакансий своего варианта использованы два условия. Поле role должно иметь значение «Программист, разработчик», а название вакансии должно содержать обозначение C или C++. Учтено написание C++ на кириллице и исключены обозначения C#, C&B и C-level. После фильтрации получено 3 676 записей.





MapReduce

Реализован подсчёт частот навыков с чтением CSV непосредственно из объектного хранилища через S3 API. В обработке использованы чанки по 1 000 отобранных записей и два процесса-маппера.
На этапе Map поле key_skills разделяется по точке с запятой. Из названий навыков удаляются крайние пробелы, пустые элементы и повторы внутри вакансии. Навыки сортируются, после чего формируются все сочетания размера 1, 2, 3.
Каждый маппер возвращает частоты сочетаний своего чанка. На этапе Reduce частоты одинаковых сочетаний суммируются.

Расчет в DuckDB

Второй способ вычисления реализован с помощью SQL-запроса DuckDB. Для независимого SQL-расчёта из RustFS загружается временная копия той же версии исходного объекта. Совпадение версии контролируется по ETag.
SQL-запрос самостоятельно фильтрует вакансии по роли и названию, разбивает поле навыков на элементы, удаляет повторы и формирует сочетания из одного, двух и трёх навыков. Частоты рассчитываются с помощью группировки.
Проверка выполнена для всех сочетаний и их частот. Результаты MapReduce и DuckDB полностью совпали.

Размер	Уникальных	Число вхождений
Один навык	1 647	16 218
Пара навыков	17 565	59 034
Тройка навыков	86 189	200 933

```
Вакансий варианта: 3676
Размер 1: уникальных 1647, вхождений 16218
Размер 2: уникальных 17565, вхождений 59034
Размер 3: уникальных 86189, вхождений 200933
Все частоты MapReduce и DuckDB совпали.
```

Рост количества пар и троек объясняется числом сочетаний навыков внутри вакансии. Если в вакансии указано k различных навыков, из неё формируются k отдельных навыков, k(k−1)/2 пар и k(k−1)(k−2)/6 троек.
Например, одна вакансия с 10 различными навыками даёт 10 одиночных элементов, 45 пар и 120 троек. Поэтому суммарное число вхождений сочетаний может существенно превышать количество вакансий.
В рассматриваемой выборке получено 16 218 вхождений отдельных навыков, 59 034 вхождения пар и 200 933 вхождения троек. Повторное указание одного навыка внутри вакансии не увеличивает его частоту, поскольку перед вычислением сочетаний повторы удаляются.

Сравнение производительности

Вычисления выполнены на компьютере с процессором AMD Ryzen 7 5700U, Windows 11, 16 ГБ. Использованы Python 3.13.3, DuckDB 2.0.0, boto3 1.44.0 и psutil 7.2.2. MapReduce запускался с двумя процессами-мапперами и размером чанка 1 000 отобранных записей.
Для каждого подхода выполнено по три запуска в отдельных процессах. Измерялись полное время выполнения процесса и максимальная наблюдаемая сумма RSS рабочего процесса и его дочерних процессов.
Время MapReduce включает чтение объекта из RustFS и вычисление частот. Время DuckDB включает загрузку объекта во временный CSV и выполнение SQL-запроса.

Попытка	Метод	Время, с	Пик памяти, МиБ
1	MapReduce	11,994	126,04
1	DuckDB	7,890	533,36
2	MapReduce	11,024	127,57
2	DuckDB	5,943	499,65
3	MapReduce	12,296	125,02
3	DuckDB	5,911	504,83

Медианное время MapReduce составило 11,994 с, DuckDB - 5,943 с. Медиана пиков памяти составила 126,04 МиБ и 504,83 МиБ соответственно. На данной конфигурации DuckDB выполнил задачу примерно в два раза быстрее, при этом потребление памяти оказалось примерно в четыре раза выше. MapReduce обрабатывает ограниченное число чанков одновременно, однако итоговый словарь частот сохраняется в памяти. DuckDB использует собственные механизмы выполнения SQL и промежуточные структуры для формирования и группировки сочетаний.

Анализ влияния навыков на заработную плату

Для сравнения использованы вакансии с зарплатой в рублях и признаком gross=false, то есть с суммой на руки.
Если указаны обе границы зарплатной вилки, оценка зарплаты рассчитывается как их среднее арифметическое. При наличии одной границы используется её значение. Пустые, неположительные и некорректные значения исключаются.
В анализ включены сочетания не менее чем с 10 зарплатными наблюдениями.
Среди пар наибольшая положительная разница относительно выбранной базы получена для Linux + OpenCV. Медианная зарплата составляет 357 500 рублей против 180 000 рублей у базового навыка Linux. Разница составляет 177 500 рублей. Возможное объяснение — специализация таких вакансий на задачах компьютерного зрения и более высокие требования к квалификации.
Для тройки C++ + PostgreSQL + Python медианная зарплата составляет 330 000 рублей против 130 000 рублей у базовой пары C++ + Python. Разница достигает 200 000 рублей. Возможная причина — сочетание требований к разработке и работе с базами данных.
Отрицательная разница получена для пары C++ + JavaScript и тройки C++ + JavaScript + ООП. Их медиана составляет 50 000 рублей, что ниже медиан соответствующих баз на 125 000 и 140 000 рублей. Возможные объяснения включают различия в уровне вакансий, регионах и составе работодателей. Для проверки этих гипотез требуется дополнительный анализ.

Сочетание	База сравнения	Наблюдений сочетания / базы	Медиана сочетания, руб.	Медиана базы, руб.	Разница, руб.
Linux + OpenCV	Linux	10 / 350	357 500	180 000	+177 500
C++ + JavaScript	C++	19 / 417	50 000	175 000	−125 000
C++ + PostgreSQL + Python	C++ + Python	10 / 50	330 000	130 000	+200 000
C++ + JavaScript + ООП	C++ + ООП	15 / 74	50 000	190 000	−140 000

```
Сочетания Linux + OpenCV и C++ + PostgreSQL + Python связаны с более высокой медианной зарплатой относительно выбранной базы.
```

Сочетания с JavaScript в приведённых примерах связаны с более низкой медианой.

Региональный анализ

Для сравнения выбраны три региона с наибольшим числом вакансий варианта - Москва, Санкт-Петербург и Новосибирск.

Регион	Всего вакансий	С указанными навыками	С подходящей зарплатой	Медианная зарплата, руб.
Москва	1 659	1 243	338	205 000
Санкт-Петербург	729	571	182	155 000
Новосибирск	172	138	56	150 000

Во всех трёх регионах лидируют отдельный навык C++, пара C++ + Linux и тройка C++ + Git + Linux.

Регион	C++, вакансий	C++ + Linux, вакансий	C++ + Git + Linux, вакансий
Москва	821	391	132
Санкт-Петербург	366	170	55
Новосибирск	104	35	15

Москва лидирует по количеству вакансий и медианной зарплате. Разрыв медиан с Санкт-Петербургом составляет 50 000 рублей, с Новосибирском - 55 000 рублей.
При меньшем количестве вакансий в Новосибирске выше доля записей с навыком C++ - 60,47% против 49,49% в Москве и 50,21% в Санкт-Петербурге. Доля пары C++ + Linux составляет 20,35%, 23,57% и 23,32% соответственно.
Совпадение лидирующих сочетаний показывает сходство наиболее востребованного технологического стека

Проверка отказа маппера

Для проверки поведения программы при отказе выполнен запуск с чтением исходного CSV из RustFS. После обработки одного чанка один из процессов-мапперов был принудительно завершён с кодом 86. Отказ внедрён отдельным тестовым скриптом.
Основной процесс обнаружил аварийное завершение маппера и завершился с ошибкой BrokenProcessPool и кодом возврата 1. Сохранённые ранее файлы частот остались без изменений. Их сохранность проверена сравнением контрольных сумм SHA-256 до и после отказа.
Затем тестовый скрипт выполнил полный повторный запуск без внедрения отказа. Программа успешно обработала 3 676 вакансий и подтвердила совпадение всех частот MapReduce и DuckDB. Контрольные суммы трёх полученных файлов полностью совпали с контрольными суммами результатов до отказа.
Проверка подтверждает обнаружение отказа и возможность восстановления результатов полным перезапуском.

```
FAULT INJECTION: mapper PID 7796 exits with code 86
```

```
concurrent.futures.process.BrokenProcessPool: A process in the process pool was terminated abruptly while the future was running or pending.
(.venv) PS C:\Users\shaki\PycharmProjects\it-vacancies-analysis> Get-Content results/rustfs/mapper_retry_log.txt -Encoding utf8
Вакансий варианта: 3676
Размер 1: уникальных 1647, вхождений 16218
Размер 2: уникальных 17565, вхождений 59034
Размер 3: уникальных 86189, вхождений 200933
Все частоты MapReduce и DuckDB совпали.
```